

Grease: A Private Payment Channel Protocol for Monero

Grease Team

January 21, 2026

Contents

1. Introduction	4
1.1. Payment channels in Monero	4
1.1.1. Enter Grease	5
1.1.2. Why does another platform have to be involved?	5
2. Design principles	7
2.1. Anti-principles	7
3. The Grease Channel Lifecycle	8
3.1. Overall design description	8
3.2. High-level state machine	9
3.3. New Channel	10
3.3.1. Channel Id	12
3.4. Establishing the Channel	12
3.5. Wallet transaction protocol	14
3.5.1. Overview	14
3.5.1.1. Phase 1: Customer Preprocessing	14
3.5.1.2. Phase 2: Merchant Preprocessing and Partial Signing	14
3.5.1.3. Phase 3: Customer Verification and Signing	15
3.5.1.4. Phase 4: Final Verification	15
3.6. Channel Updates	17
3.7. Co-operative Close	19
3.7.1. Channel Close messages	19
3.8. Dispute resolution	20
4. The Arbiter	21
4.1. What the arbiter is for	21
4.2. Custody versus authority	21
4.3. Attestation as a decryption key	22
4.4. Gating a close on the true latest state	24
4.5. The binding proof	25
4.6. The dispute state machine	28
4.7. The dispute in practice	30
4.8. What is public, and to whom	31
4.9. Trust and collusion	32
4.10. Instantiation on the Internet Computer	33
4.11. Optional: fees, deposits, and cleanup	34
4.12. Possible extensions	35
4.13. Summary	36
5. Future extensions and known limitations	37
5.1. Bilateral channels only	37
5.2. Trust in the arbiter	37
5.3. Liveness depends on cooperation or an extension	37
5.4. Watching duty	37
6. Nomenclature	39
6.1. Symbols	39

6.2. Subscripts	39
References	40
Index of Figures	41
Index of Tables	41
Index of Listings	41

1. Introduction

Monero is, alongside cash, the world's most private¹⁻³, and, arguably the best currency in circulation, but the user *experience* remains less than ideal. This comment is not necessarily aimed at user *interfaces* – for example, there are Monero wallets that are very attractive and easy to use – but the fundamental design of Monero means that:

- many, especially new, users find they can make only one payment roughly every 20 minutes when their wallet holds a single spendable output (the change from the first payment typically requires about 10 confirmations before it can be spent again),
- due to the absence of scripting capabilities, use-cases that capture the public imagination, like DeFi, are not possible in vanilla Monero.

Therefore, the *experience* of using Monero tends to be one of waiting, and limited functionality.

For Monero to achieve mass adoption, it will need to find ways to:

- provide an order of magnitude *better UX* (again, not necessarily UI). Locking UTXOs after spending and block confirmation times add significant friction to Monero and is a turn-off for new users who are already unsure about how cryptocurrency works.
- provide *instant confirmations* when purchasing with Monero.
- enable *seamless point-of-sale transactions* so that using Monero for purchases feels no different to using a credit card or Venmo.
- enable DeFi for Monero. DeFi is the future of finance. The lack of permissionless access to bank-like services (loans, insurance, and investments) is a key barrier to truly democratic money.
- provide for Monero-backed and/or privacy-maximizing stable coins.

A payment-channel solution for Monero is one of the foundational requirements for achieving these goals in Monero. The other is smart contracting functionality, but that is out of scope for this project.

1.1. Payment channels in Monero

Monero's primary function is private, fungible money. This goal very likely excludes any kind of meaningful on-chain state management for Monero, since state implies heterogeneity. And heterogeneity immediately breaks fungibility. That's not to say that some hitherto undiscovered insight won't allow this in future, but for the short and medium-term at least, any kind of state management for Monero transactions or UTXOs would have to be stored off-chain.

The small amount of state a channel needs in order to resolve a dispute — which of two signed records is the more recent — is not private; it is a public fact that both parties already share. What it needs is a platform that will evaluate that fact deterministically and act on it in a way no single operator can subvert. A public consensus platform provides exactly this.

It's the goal of this project to pair Monero (for private money) with such a platform (for trustless dispute resolution) to create a proof-of-concept payment channel for Monero.

1.1.1. Enter Grease

The Grease protocol is a new bi-directional payment channel design with unlimited lifetime for Monero. FCMP++ (Full-Chain Membership Proofs++) replaces Monero's fixed-size ring signatures with a succinct proof that a spent output belongs to the whole set of outputs on the chain; because the anonymity set is chosen only when a transaction is finally broadcast, a channel's pre-signed closing transaction never becomes stale — which is what gives a Grease channel its unlimited lifetime.

Using the Grease protocol, two peers may trustlessly¹ cooperate to share, divide and reclaim a common locked amount of Monero XMR while minimizing the online transaction costs and with minimal use of outside trusted third parties.

The Grease protocol maintains all of Monero's security. No identifiable information about the peers' privately owned Monero wallets is shared between the peers. This means that there is no way that privacy can be compromised. Each channel lifecycle requires two Monero transactions, with effectively unlimited near-instant updates to the channel balance in between these two transactions. This dramatically improves the scalability of Monero.

The Grease protocol is based on the original AuxChannel⁴ paper and Monet⁵ protocol. Monet provides a sketch of a payment channel design but many details were omitted or elided. Grease is a full-fledged implementation with full consideration given to state management, multiple concurrent channels, peer-to-peer communication and encryption and practical performance.

Every update and the final closure of the channel require an online interaction between channel parties. In order to prevent the accidental or intentional violation of the protocol by a peer not interacting and thus jamming the channel closure, Grease introduces an external *arbiter*. The arbiter needs to run on a stateful, logic- and time-aware platform whose behavior is determined by consensus rather than by any single operator. A decentralized smart-contract platform satisfies this requirement.

1.1.2. Why does another platform have to be involved?

Offline payment channels necessarily *require* a state management mechanism. Typically, the scripting features for a given blockchain allow for this state to be managed directly. However, Monero's primary design goals are privacy and fungibility. Attaching state to UTXOs would create a heterogeneity that threatens these goals. Fungibility is more important than features when it comes to privacy.

The dispute state does not have to be managed on the same chain though. Any place where it is:

- available,
- reliable,
- has trust assumptions appropriate for the value involved,

¹No trust is needed between the channel parties. Dispute resolution relies on an arbiter, whose trust assumptions reduce to those of its consensus platform; the arbiter holds no funds and no secret whose leak could cost a channel anything.

will suffice. Note that this state is not secret — it is a public record of which channel update is the most recent — so the platform is never asked to keep a secret, only to run its published logic honestly.

The initial implementation targets the Internet Computer, whose canisters can hold a stable threshold key on behalf of the whole validator set and release an attestation on demand under the control of on-chain logic.

The arbiter acts as a third-party judge in disputes, but holds nothing that could be stolen. At every update, each peer encrypts a fresh secret offset so that it can only be recovered once the arbiter attests that this state is the channel’s latest. This uses *identity-based encryption*: a value can be sealed to a short “identity” — here the statement “this is the channel’s latest state” — using only a public key, and it becomes decryptable only when the arbiter issues the single attestation that matches that identity. The offset is sealed long before any dispute; if one arises, the arbiter attests the most recent state either party can prove, and that attestation is exactly the key that unseals the offset closing at it. A stale or fabricated state is never attested and can never be closed. Section 4 has full details on the arbiter’s design and implementation.

2. Design principles

Grease is a bidirectional two-party payment channel. This means that funds can flow in both directions, but in the vast majority of cases, funds will flow from one party (the “customer”) to the other (the “merchant”). This designation is somewhat arbitrary, either party in a channel may assume either role. Most channels will have participants that are easily identifiable in either the customer or merchant role and we use that terminology throughout this document.

Grease embraces this use case and optimizes the design and UX based on the following assumptions:

- The merchant is, by convention, the party that interacts with the arbiter — lodging any optional setup deposit and covering its fees.
- The merchant therefore needs to hold a small amount of the arbiter chain’s native token to pay those fees.
- The customer needs a small quantity of the arbiter chain’s token to dispute a closure. In the vast majority of cases this won’t be necessary, since funds almost always flow in one direction from the customer to the merchant. In practice, the merchant can supply this if the customer has none. The customer must watch the arbiter and present a more recent cross-signed record than the one the merchant tried to close on in the case of a dispute.
- In the vast majority of cases, the customer opens a channel with m XMR and the merchant starts with a zero XMR balance (since the merchant is providing assets or services in exchange for Monero).
- Usually, both parties mutually close the channel. Either party *may* close the channel unilaterally through the arbiter, and are able to claim their funds once the adjudication window has elapsed. In this case, the party doing so is usually the merchant since they have the greater incentive to do so in the case of an abandoned channel.

2.1. Anti-principles

The following design goals are explicitly *excluded* from the Grease design:

- Multi-hop channels. Multi-hop channels are probably *possible* in Grease, but they are not a design goal.

Taking the Lightning Network as the case study, CJ argues⁶ that the vast majority of the utility of lightning is captured by bilateral channels, with a tiny fraction of the complexity.

3. The Grease Channel Lifecycle

3.1. Overall design description

Grease builds on the Monet⁵ design, a payment channel protocol that introduced the use of a third-party arbiter to resolve channel disputes.

A Grease payment channel is a 2-party bidirectional channel. The most common use case is in a multi-payment arrangement between a customer and a merchant, and so we will label the parties as such.

To set up a new channel, the customer and merchant agree on the funds to be locked in the channel. They're usually all provided by the customer, but it doesn't need to be. These funds are sent to a new 2-of-2 multisig wallet, which is created on the Monero blockchain for the sole purpose of serving the channel.

The idea is that a *commitment transaction*, so-called for reasons that will be made clear later, spends the funds out of the multisig wallet back to the customer and merchant can be trustlessly, securely and rapidly updated many thousands of times by the customer and merchant without having to go on-chain.

Every time the channel is updated, the customer and merchant provide signatures that *can't be used to spend the funds* out of the multisig wallet, but *prove* that they will be able to spend the funds if a small piece of missing data is provided². When the channel is closed, the customer gives the merchant that little piece of data and the funds are spent out of the multisig wallet to the customer and merchant, closing the channel.

If the merchant cheats and tries to close the channel with an outdated state, or decides not to broadcast the commitment transaction, the customer can resolve the closure through an *arbiter*.

The arbiter resolves disputes. It is a deterministic state machine deployed on a public consensus platform, and it holds no share of the channel's funds and no secret whose leak could cost a channel anything. It won't be called upon for the vast majority of channel instances, but its presence is what makes cheating pointless.

Note

In fact, you could run Grease without an arbiter, if there is a high-trust relationship between the customer and merchant.

At every channel update, each party generates a fresh secret offset that completes the counterparty's half of that state's closing signature, and hands the counterparty a *verifiably encrypted offset*: the offset, encrypted so that it can only be recovered once the arbiter attests that this state is the channel's latest. No secret is ever handed to the arbiter, and the arbiter never sees a ciphertext.

²These signatures are called adapter signatures.

If the merchant tries to close using an outdated state, or refuses to publish anything at all, the customer presents its latest cross-signed record to the arbiter. The arbiter opens a window, during which either party may present a newer record, and then attests the highest update count it has seen.

That attestation releases the offset for the *true latest* state, and only that state. The customer completes and broadcasts the latest closing transaction, recovering its correct balance; the merchant's stale close is never attested and so can never be completed. Cheating is not punished — it is simply frozen — which is exactly what motivates parties to behave honestly.

Warning

Once a channel is closed, neither party should use the 2-of-2 multisig wallet again, since there exists another party that can immediately spend out of that wallet.

3.2. High-level state machine

On a high level, the payment channel lifecycle goes through 6 phases:

- **New** - The channel has just been created and is entering the establishment negotiation phase. Basic information is swapped in this phase, including the public keys of the peers, the nominated arbiter, and the initial balance. The channel parameters agreed here — initial balances, public keys and closing addresses — feed the channel id, which is finalized once the funding output exists and commits to it (See Section 3.3.1). If both parties are satisfied with the proposed channel parameters, the channel moves to the **Establishing** state.
- **Establishing** - The channel is being established. This phase creates and funds the multisig wallet. Once both parties have verified the funding transaction, the parties will share an `AckChannelEstablished` message. Once acknowledged, an `OnChannelEstablished` event is emitted, and the channel will move to the **Open** state.
- **Open** - The channel is open and ready for use. Any number of channel update events can occur in this phase and the channel can remain in this state indefinitely. The channel remains in this state until the channel is closed via the amicable **Closing** state or the **Disputing** state. At each update, both parties cryptographically *commit* to the new state of the channel and collaborate to produce a new, partially-signed commitment transaction.
- **Closing** - The channel is being closed. This phase includes sharing of adapter secrets and signing of the final commitment transaction. Once both parties have signed the final commitment transaction, any party will be able to broadcast it, but by convention it will be the merchant that does so.
- **Closed** - The channel is closed. A channel can reach the **Closed** state after a co-operative close, following a resolved dispute, or after several error conditions arising during the **New** and **Establishing** phases. The arbiter keeps no per-channel state outside an open dispute, so there is nothing to clean up.
- **Disputing** - The channel is being disputed because someone initiated a unilateral close. If the local party initiated it, this phase includes presenting its latest cross-signed record to the arbiter and waiting for the adjudication window to expire so that the counterparty's offset can be recovered in order to synthesize the closing transaction. If the other party initiated the dispute, we can present a newer record to the arbiter, or do nothing if we hold no newer

state. The final state transition is always to the `Closed` state, only the reason can vary. See Section 4 for details.

3.3. New Channel

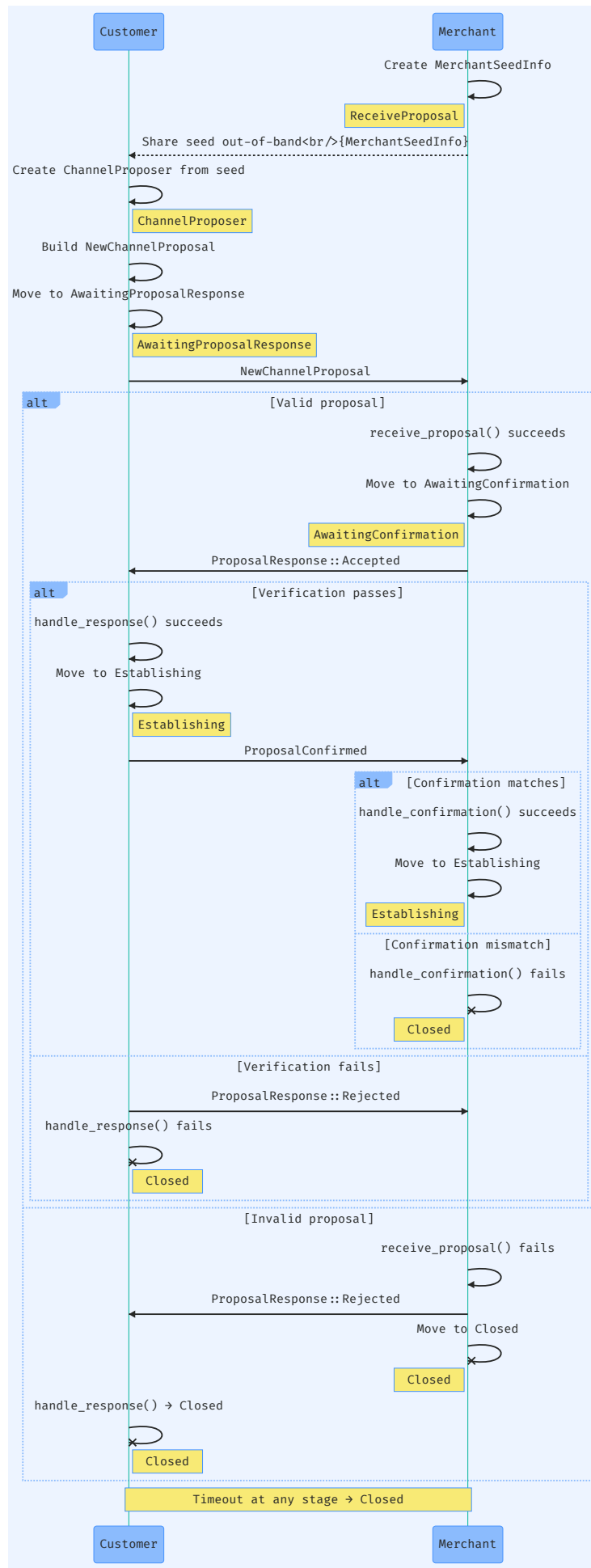
A new channel is established when a Merchant shares some initialization data with a Customer out-of-band.

The customer takes this data, combines it with their own information, and sends a channel proposal to the Merchant.

There are **three** half-rounds of communication in this phase³:

1. Out-of-band channel initialization data (CID) sharing from Merchant to Customer:
 - Contact information for the merchant
 - Channel seed metadata. This includes metadata so that both merchant and customer can uniquely identify the channel throughout the channel's lifetime. This includes:
 - a random nonce id,
 - the merchant's closing address,
 - the requested initial balances,
 - the merchant's public key,
 - the dispute window duration
 - Protocol-specific initialization data. This might include commitments for parameters that will be shared later, the arbiters that can be accepted, etc.
2. New channel proposal from Customer to Merchant
3. Accepting the proposal from Merchant to Customer

³See `server.rs:customer_establish_new_channel`



Listing 1: The New Channel proposal sequence
THE GREASE CHANNEL LIFECYCLE

3.3.1. Channel Id

The channel id uniquely identifies the channel and *commits to its funding output*, so that every record and every dispute is bound to one specific 2-of-2 wallet. It is the prefix “XGC” followed by the first 31 bytes of the **Blake2b-512** hash of the transcript below, rendered in hexadecimal (a 65-character string).

The transcript combines the negotiated channel metadata with the funding output’s *linking tag* L_F — the key image that any spend of the funding output publishes, which the two parties compute jointly from the shared wallet:

- The merchant public key, P_B , 32 bytes in little-endian byte order,
- The customer public key, P_A , 32 bytes in little-endian byte order,
- The merchant initial balance in piconero, as a 64-bit unsigned integer in little-endian byte order,
- The customer initial balance in piconero, as a 64-bit unsigned integer in little-endian byte order,
- The nominated closing address of the merchant, as a Base58 string,
- The nominated closing address of the customer, as a Base58 string,
- The funding output linking tag, L_F , 32 bytes in little-endian byte order,
- A merchant nonce, a 64-bit little-endian unsigned integer, randomly chosen by the merchant, and
- A customer nonce, a 64-bit little-endian unsigned integer, randomly chosen by the customer.

Because L_F is only fixed once the shared funding wallet exists, the id is *finalized during establishment* (Section 3.4), not during the initial negotiation: the parties settle the metadata in the **New** phase and compute the id as soon as they have jointly derived L_F .

Two properties follow, both relied on by the arbiter (Section 4). First, the id *commits to* L_F , so a record or attestation carrying it is bound to this funding output and cannot be replayed against another channel. Second, the id *hides* L_F : the two random nonces blind the hash, so publishing the id reveals neither the funding output nor any way to locate its eventual close on the Monero chain — only a holder of the transcript (the counterparties, and any watchtower they appoint) can open the commitment. The arbiter uses the id only as an opaque, unique label, so this binding costs it no knowledge of the channel’s funds.

3.4. Establishing the Channel

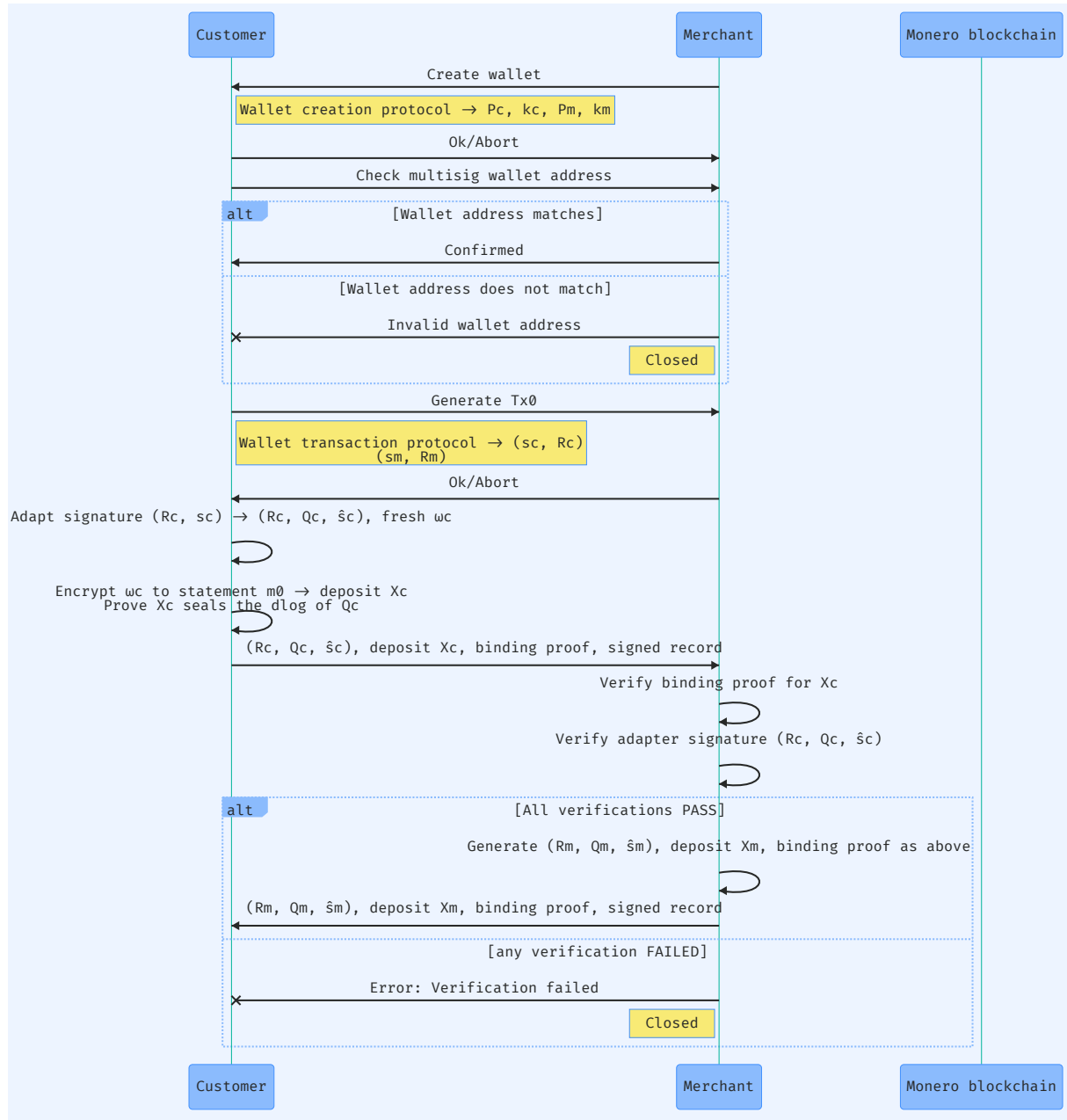
Establishing a channel to accept payments requires the following preparatory steps:

1. Both parties collaboratively create a new shared multisig wallet to hold the channel funds.
2. The parties derive the channel id, which commits to the funding output, and agree on the arbiter to be used for disputes (Section 4).
3. Each party watches the Monero blockchain for the funding transaction to confirm it has been included in a block.
4. The parties prepare the initial channel state: each adapter-signs the counterparty’s closing transaction with a fresh secret offset, and hands the counterparty a *verifiably encrypted offset* — that offset encrypted to the statement that this is the channel’s latest state.

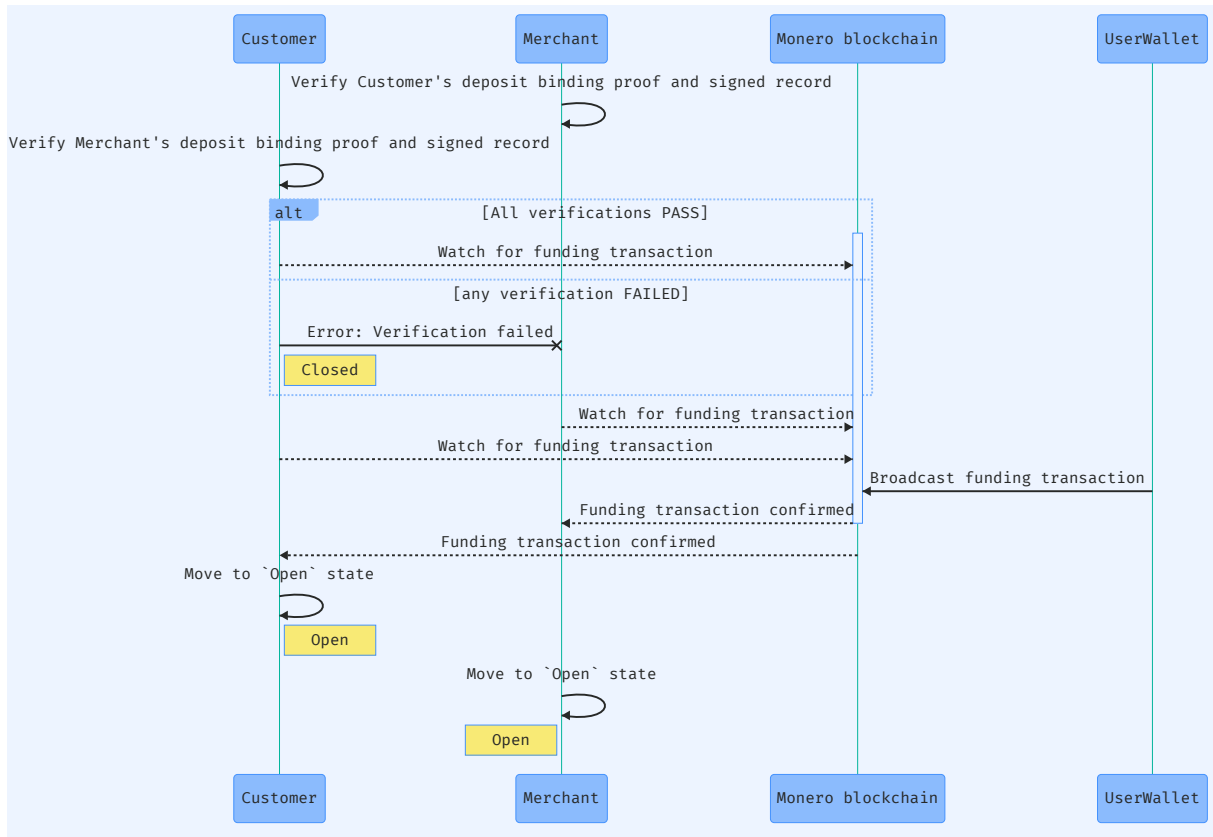
5. Each party verifies the binding proof accompanying the counterparty's verifiably encrypted offset before accepting the initial state.
6. The customer funds the multisig wallet with the agreed initial balance.
7. Once the funding transaction is confirmed, the channel is open and ready to use.

Note

No SNARKs are required to establish or operate a channel. The arbiter is not contacted during establishment; it holds no state for a channel until a dispute is opened.



Listing 2: Establishing a new Channel



Listing 3: Establishing a new Channel, continued

3.5. Wallet transaction protocol

3.5.1. Overview

Grease uses 2-of-2 multisig transactions generated using FROST (Flexible Round-Optimized Schnorr Threshold signatures) combined with adapter signatures. The roles of customer and merchant are interchangeable, but for simplicity we will refer to the customer as the party initiating the protocol.

3.5.1.1. Phase 1: Customer Preprocessing

The Customer initiates the protocol by generating preprocessing data based on transaction details. This preprocessing step is fundamental to FROST and involves:

- Creating commitments for the signing process
- Generating nonce values that will be used during signature creation
- Producing data (denoted as C^C) that encapsulates these commitments.

The Customer then transmits both the preprocessing data and the transaction details to the Merchant.

3.5.1.2. Phase 2: Merchant Preprocessing and Partial Signing

Upon receiving the Customer's data, the Merchant carries out:

1. **Generates its own preprocessing data** (C^M) using the same transaction details
2. **Creates a partial signature** on the transaction using:
 - The transaction details

- Its own preprocessing commitments
 - This produces a partial signature, (R_0^M, s_0^M) .
3. **Adapts the signature** by converting the partial signature into an adapter signature format:
- Produces an adapted signature tuple $(R_0^M, Q_0^M, \hat{s}_0^M)$
 - For every state — the initial one and each subsequent update — it generates a fresh random witness value (ω_0^M for the initial state) that serves as the secret offset for that state.

The adapter signature is a cryptographic construct that allows the Merchant to create a valid-looking signature that is “locked” by a secret witness value. This signature appears complete but cannot be verified as a standard signature without knowledge of the witness.

The Merchant transmits its preprocessing data and the adapted signature back to the Customer, but notably **withholds** the witness value ω_0^M .

3.5.1.3. Phase 3: Customer Verification and Signing

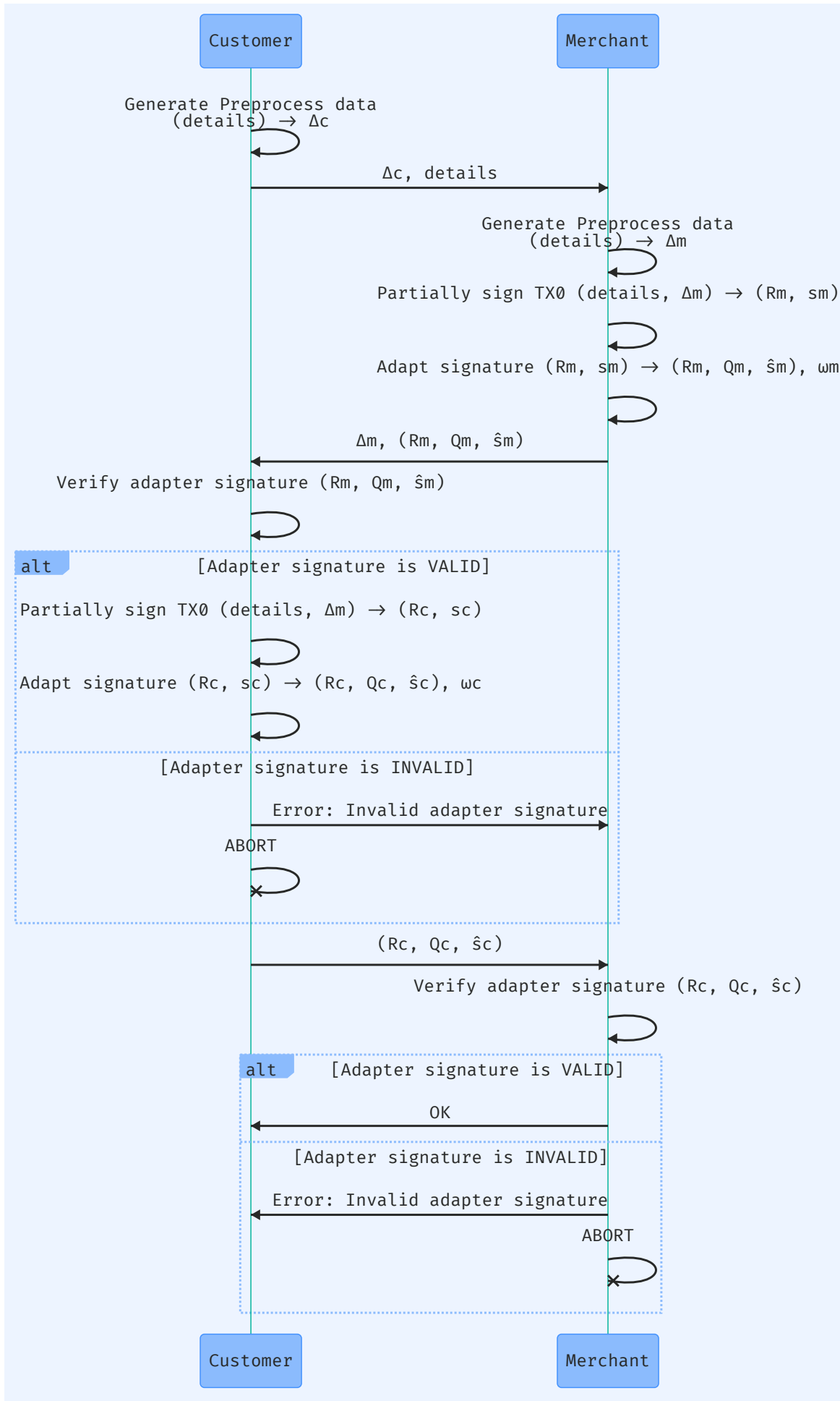
The Customer now performs verification and, if successful, creates its own adapted signature:

- **Verifies the adapter signature** $(R_0^M, Q_0^M, \hat{s}_0^M)$ using the Merchant’s public key
- If verification succeeds:
 - Creates its own partial signature (R_0^C, s_0^C) on the transaction
 - Adapts this signature to produce $(R_0^C, Q_0^C, \hat{s}_0^C)$ and witness ω_0^C
 - Transmits the adapted signature to the Merchant
- If verification fails:
 - Sends an error message to the Merchant
 - Aborts the protocol

3.5.1.4. Phase 4: Final Verification

The Merchant receives the Customer’s adapted signature and performs the final verification:

- **Verifies the adapter signature** $(R_0^C, Q_0^C, \hat{s}_0^C)$ using the Customer’s public key
- If verification succeeds:
 - Sends confirmation to the Customer
 - Both parties now hold valid adapted signatures
- If verification fails:
 - Sends an error message to the Customer
 - Aborts the protocol



3.6. Channel Updates

Once a channel is open the parties may transact and update the XMR balance between themselves. This is done entirely off-chain, and happens near-instantaneously.

Note

Under FCMP++, the membership (decoy) set for a spend is chosen only when a transaction is finally broadcast⁷, rather than when it is signed. This has two consequences for channel updates:

- An update never needs to query the Monero blockchain to select decoys, a significant performance improvement.
- A channel can stay open indefinitely: its pre-signed closing transaction can never become stale.

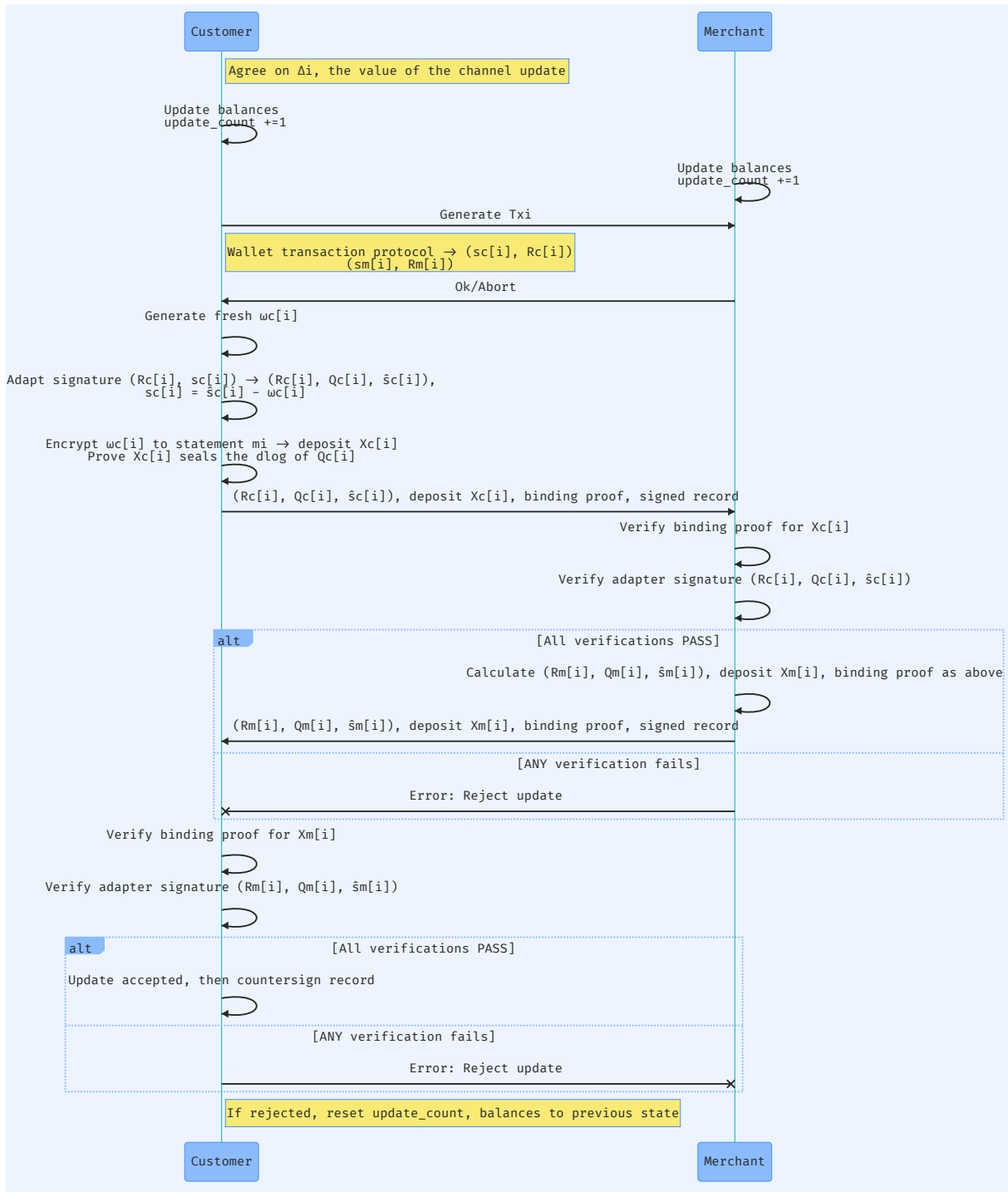
Either party can initiate a channel update. Since it is usually the merchant, we designate them as the initiator of the update, but the labels can be switched without loss of generality.

The merchant decides on the delta value, which can be positive (customer pays merchant) or negative (merchant refunds customer). He then carries out phase 1 on the transaction protocol to create a new Monero transaction reflecting the updated balances.

1. Privately, generate a fresh random offset ω_i for the counterparty's copy of the new state, and adapter-sign the counterparty's commitment transaction with it.
2. Encrypt ω_i to the statement $m_i = (\text{id}, i)$ under the arbiter's public key, producing a *verifiably encrypted offset* for the counterparty together with a binding proof that the ciphertext seals the offset of the adapter point used in this state (Section 4.3).
3. Cross-sign the update record (id, i, h_i) , where h_i is the canonical hash of the new state's closing transaction.
4. Send the update package — the adapter signature, the verifiably encrypted offset and its binding proof, and the signed record — to the peer.

Verification consists of:

1. Verify the adapter signature on the local party's commitment transaction.
2. Verify the binding proof on the counterparty's verifiably encrypted offset.
3. Verify the counterparty's signature on the update record, and countersign it.



Listing 5: Updating a Channel

Note

The actual implementation may streamline communication rounds by batching messages together. In particular, the verifiably encrypted offset and update-record messages can be sent together with the transaction protocol messages to reduce the total number of round trips.

Each state's offset is generated independently at random, so no offset reveals anything about any other. A party that later closes the channel at the agreed latest state learns nothing that would let it complete an *earlier* state, and a stale state can never be closed because the arbiter

will never attest its statement (Section 4). There is therefore no need for the offsets to chain to one another, and no zero-knowledge circuit is evaluated on an update — a channel update is a handful of Schnorr operations plus one verifiably encrypted offset and its binding proof. The proof is essentially the whole of an update’s bandwidth: 23,641 bytes at the parameters pinned in Section 4.5 (Table 1), against well under a kilobyte for the adapter signature, the record and its two signatures combined.

3.7. Co-operative Close

Either party can initiate a co-operative close of the channel at any time by sending their latest adapter offset to the counterparty in a `RequestChannelClose` message. As an illustration, we will assume that the customer initiates the close.

When the merchant receives the offset, it verifies it, by trying to complete the closing transaction signature for the expected update count. It then broadcasts the transaction to the network. The channel is then marked as `Closed`. If the closure was successful, the merchant responds with an `ChannelCloseSuccess` message. If any errors occur, the merchant responds with a `RequestCloseFailed` message, and the channel remains `Open`. If the merchant did not provide the transaction id in its response, the customer may use the provided offset to reconstruct and broadcast the closing transaction herself.

If a party becomes unresponsive during the co-operative close process, one may initiate a unilateral close through the arbiter, as described in Section 4.

3.7.1. Channel Close messages

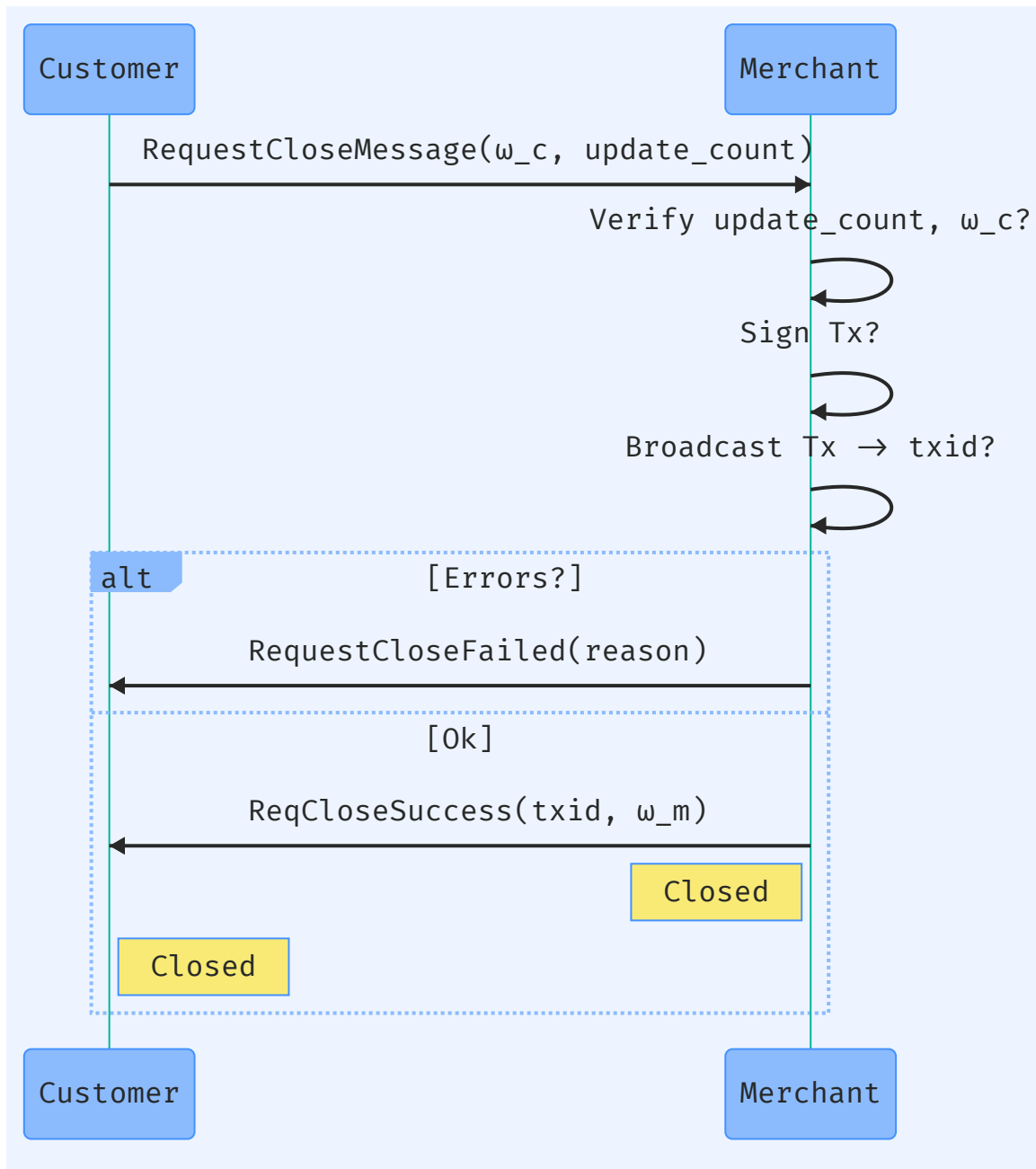
A party sends a `RequestChannelClose` message to initiate a co-operative channel closure.

```
pub struct RequestChannelClose {
    /// The globally unique channel id
    channel_id: ChannelId,
    /// The latest adapter offset for the initiating party
    offset: Scalar,
    /// The update count corresponding to the latest adapter offset
    update_count: u64,
}
```

The counterparty responds with either a `RequestCloseFailed` or `ChannelCloseSuccess` message.

```
pub struct ChannelCloseSuccess {
    /// The globally unique channel id
    channel_id: ChannelId,
    /// The latest adapter offset for the responding party
    offset: Scalar,
    /// The transaction id of the closing transaction. Optional.
    txid: Option<TxId>,
}

pub struct RequestCloseFailed {
    /// The globally unique channel id
    channel_id: ChannelId,
    /// An error code indicating the reason for the failure
    reason: CloseFailureReason,
}
```



Listing 6: The Channel Close sequence

3.8. Dispute resolution

When cooperation breaks down — a peer becomes unresponsive, or one party tries to close the channel with an outdated state — the aggrieved peer presents its latest cross-signed record to the arbiter. The arbiter opens an adjudication window, lets the counterparty answer with any newer record, and at the window’s close attests the statement for the highest update count it has seen. That attestation unseals exactly the offset that closes the channel at the true latest state; a stale close is never attested, and so can never be completed.

The dispute process, the arbiter’s state machine, and the guarantees it provides are described in full in Section 4, with the presentation flow in Section 4.7.

4. The Arbiter

4.1. What the arbiter is for

A Grease channel keeps its funds in a 2-of-2 multisig output that neither party can spend alone, and every channel state is a fully pre-signed transaction that closes the channel at that state's balances — deliberately left incomplete by a secret offset so that it cannot be broadcast until the offset is supplied. Because neither party can broadcast a commitment on its own, there is no race to rush a stale state onto the chain. Two situations remain that the channel cannot resolve for itself, and resolving them is the arbiter's entire job:

Liveness release the counterparty disappears. The honest party wants to close at the latest agreed state but cannot, because it is missing the offset that completes the counterparty's half of the closing signature. Something must release that offset on request.

Cheat adjudication a party claims that some *old* state is the latest, hoping to close at balances that favor it. Something must recognize that a newer state exists and make sure only the newer state can ever be closed.

Both reduce to one task the Monero base layer cannot perform: decide, given two cross-signed records, *which is newer*, and release exactly the offset that closes at it. The arbiter performs it. The rest of this chapter is about giving the arbiter that authority **while letting it hold no secret whose leak could cost a channel its funds**. It never holds a share of the funding key, never learns a channel's balances, and stores no per-channel secret. It is a small, deterministic state machine whose only power is to attest a public statement — and that attestation is exactly what unseals the correct offset.

Note

The arbiter is never touched on the common paths. Channel updates are peer-to-peer, and a cooperative close is a single ordinary Monero transaction (Section 3.7). The arbiter is consulted only when a party goes silent or tries to cheat.

4.2. Custody versus authority

It helps to separate two things a third party might do, because keeping them apart is the whole idea.

Custody holding a secret whose leak is catastrophic. If a third party holds a value that can complete a closing signature for a favorable state, then whoever obtains that value can drain the channel. A design built on custody is only ever as safe as the promise that the secret never leaks — and on a public, auditable platform that promise is very hard to keep.

Authority deciding, in public, which public statement to certify. Choosing “state *n* is the latest” from the records the parties have already exchanged leaks nothing. It is a pure function of information both parties already hold.

A public smart-contract chain is excellent at authority and structurally poor at custody. It can *run code* verifiably: place the logic on-chain and consensus guarantees that this logic, and only this logic, executed. What it cannot do is *store a secret that its own validators cannot see*. On a

transparent chain all state is public. On a zero-knowledge chain, “private” state is encrypted to some party who holds the decryption key — so the secret still exists somewhere, and its leak is exactly the failure a custody-based design exists to prevent.

The arbiter sidesteps this by storing **no per-channel secret at all**. What it needs from its platform is only two things:

1. **verifiable deterministic execution** — the published state machine, and nothing else, runs; and
2. **a threshold key held by the validator set itself** — used not to store channel secrets, but to **sign a statement on demand**, where the signature on a statement is exactly the key that unseals that statement’s offset.

The second ingredient is what makes this practical, and it is available today as a production service (Section 4.10). A validator set can jointly hold a single threshold-BLS master key — no member knows it in full, and a Byzantine quorum can sign with it — exposing one operation: given an application-chosen byte string m , and given that the on-chain logic authorizes it, release the group’s BLS signature on m . That signature is the arbiter’s attestation of m ; as Section 4.3 shows, it is also an identity-based decryption key for m . The master key is *global* and *application-agnostic*: it is not created per channel, holds no channel’s funds, and is shared across every application on the platform. Its compromise is therefore not a quiet, per-channel theft but a platform-wide, publicly evident event — the same risk profile as a compromised public randomness beacon.

So the arbiter keeps **authority** — which statement is true, decided by public code — and sheds **custody** — any secret whose leak matters. The residual trust is not “a custodian will not leak my key”; it is “a public validator set runs its published logic and does not collude beyond its fault threshold”. That is close to the trust the platform already demands of every application it hosts, with one qualification: the arbiter concentrates it. Its funds rest on the unforgeability and confidentiality of *one* validator subnet’s threshold key, and on governance over that subnet, rather than on an ecosystem-wide average — so the incentive to corrupt exactly that subnet is higher than for an application that only stores public state.

4.3. Attestation as a decryption key

The mechanism that turns an attestation into a release is identity-based encryption built from a threshold-BLS committee. Let the arbiter committee hold a threshold-BLS key over a pairing-friendly curve with additive groups $\mathbb{G}_1, \mathbb{G}_2$, target group $\mathbb{G}_T$, generators G_1, G_2 , and a pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$. The committee’s master secret is a scalar z that no member knows in full; the master public key

$$Z = z \cdot G_2 \tag{1}$$

is published once and is stable for the committee’s lifetime. Let $H_P(\dots)$ hash a byte string to a point in $\mathbb{G}_1$ and $H_F(\dots)$ hash a $\mathbb{G}_T$ element to a scalar in $\mathbb{Z}_\ell$, where ℓ is the prime order of the Ed25519 group on which offsets live. The pairing groups $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ share a prime order q , and we fix the mask modulus $N = \ell$ throughout — Section 4.5 shows why identifying the two, rather than reducing offsets modulo q , is what makes the binding proof cheap.

`EncryptToStatement` . Z is the arbiter's master public key; m is the statement; ω the offset.

```

1  $(U, c) = \text{EncryptToStatement}(\omega, m, Z)$ 
2 Select a random scalar  $r$ ,  $0 < r < q$  (the pairing order, not the mask modulus  $N$ ).
3 Calculate:
4    $U = r \cdot G_2$ 
5    $y = e(H_P(m), Z)^r$  in  $\mathbb{G}_T$ 
6    $s = H_F(y)$ 
7    $c = (\omega + s) \bmod N$ 
8 Return  $(U, c)$ .
```

Figure 1: Encrypt an offset to a statement.

For any statement m , the committee's BLS signature on m is

$$\sigma_m = z \cdot H_P(m) \in \mathbb{G}_1. \quad (2)$$

Producing σ_m needs a quorum; verifying it is the pairing check $e(\sigma_m, G_2) = e(H_P(m), Z)$. The key observation is that this same σ_m is an *identity-based decryption key* for the identity m . A party can encrypt a scalar offset ω against the statement m using the arbiter's *stable* public key Z , long before anyone has signed m :

Correctness follows from bilinearity: once the committee releases $\sigma_m = z \cdot H_P(m)$,

$$e(\sigma_m, U) = e(z \cdot H_P(m), r \cdot G_2) = e(H_P(m), G_2)^{zr} = e(H_P(m), Z)^r = y, \quad (3)$$

so the decryptor recovers the same mask y and hence ω . The ciphertext can be formed by anyone at any time against the stable Z ; decryption becomes possible **exactly when** the committee attests m , and not before.

The property the whole design rests on is a corollary: **whoever decides which m gets attested decides which ciphertexts can ever open**. If the arbiter attests m only when its deterministic logic says m is true, then ciphertexts addressed to false statements stay sealed forever — with no custodian, no per-message secret, and no way for any single committee member to release anything alone.

`DecryptWithAttestation` . σ_m is the arbiter's attestation of m .

```

1  $\omega = \text{DecryptWithAttestation}(U, c, \sigma_m)$ 
2 Calculate:
3    $y = e(\sigma_m, U)$  in  $\mathbb{G}_T$ 
4    $s = H_F(y)$ 
5    $\omega = (c - s) \bmod N$ 
6 Return  $\omega$ .
```

Figure 2: Decrypt an offset once the statement has been attested.

4.4. Gating a close on the true latest state

Recall that each channel state's unilateral close is gated by a secret offset: to complete the counterparty's half of the closing signature for state i , a party needs that party's offset ω_i , whose public point is $Q_i = \omega_i \cdot G$ on Ed25519.⁴ We bind that offset to a statement.

At each update to state i , the parties agree the statement

$$m_i = (\text{id}, i) \quad (\text{on channel } \text{id}, \text{ state } i \text{ is the latest}), \quad (4)$$

where `id` is the channel identifier (Section 3.3.1), which commits to the funding output. Each party generates a *fresh* offset ω_i for the counterparty's state- i close, and hands the counterparty an `EncryptToStatement` ciphertext of that offset addressed to m_i — a **verifiably encrypted offset**. Because it is worthless unless it is honest, it travels with a non-interactive proof — constructed in Section 4.5 — establishing, as a single statement, that

1. (U, c) is a well-formed ciphertext under the arbiter's stable key Z , addressed to **exactly** m_i ; and
2. the offset sealed inside decrypts to the discrete logarithm of the **very adaptor point** Q_i used in the counterparty's held pre-signature for state i .

The receiving party verifies this proof before it treats the update as complete. Offsets are freshly generated and domain-separated per (channel, state, party); reusing one would let a single attestation unseal ciphertexts across many channels at once.

Note

The arbiter never sees a verifiably encrypted offset. Ciphertexts live only with the two parties. The arbiter's release is the public value σ_m , and whoever holds the matching ciphertext can then decrypt it — exactly as anyone can decrypt a message timelocked to a randomness beacon once that beacon publishes the round.

Putting the pieces together explains why cheating cannot pay. To complete a close at state i a party needs ω_i , sealed under m_i , unsealable only if the arbiter attests m_i ; and the arbiter (Section 4.6) attests only the statement at the channel's high-water mark. Therefore:

- an **honest** close at the latest state n opens, because the arbiter attests m_n ;
- a **stale** close at state $i < n$ never opens, because m_i is never attested once a newer record has been presented — its ciphertext stays sealed;
- a **fabricated** state has no verifiably encrypted offset and no cross-signed record, so there is nothing to attest and nothing to open.

The cheat is not punished; it is **frozen**. No revocation secret, no penalty transaction, and no collateral are needed for the adjudication to be safe.

⁴Under FCMP++ the spend proof checks a signature against two bases at once, so the offset is applied to both the standard generator and the re-randomized key-image base; the encrypted value is the same scalar ω_i , and the account here is unchanged.

4.5. The binding proof

The two requirements just stated — that (U, c) is well-formed under Z and addressed to exactly m_i , and that the sealed offset is the discrete logarithm of the adaptor point Q_i the recipient already holds — are met by one non-interactive proof, produced by the encryptor and checked before the update is treated as complete. It carries a hard confidentiality constraint: it must reveal **nothing** about the offset $\omega = \omega_i$. The encryptor generated ω for the counterparty’s own state- i close and adaptor-signed that close with it, so an ω learned at update time would let the recipient complete that close at once — including one state later, when it is stale. The proof is therefore zero-knowledge in ω .

With the mask modulus fixed at $N = \ell$ (Section 4.3), ω is a native Ed25519 scalar and the recipient computes, for free,

$$S := c \cdot G - Q_i = (\omega + s) \cdot G - \omega \cdot G = s \cdot G, \quad (5)$$

since G has order ℓ and reducing modulo ℓ leaves no carry in the exponent. So $s \cdot G$ is pinnable — yet $s = H_F(y)$ is a random-oracle hash of the pairing output, and no Σ -protocol can cross that hash to tie $s \cdot G$ back to $U = r \cdot G_2$. Every efficient “the plaintext is this discrete logarithm” proof relies on the ciphertext being an *algebraic* function of the plaintext, and the hash mask destroys that structure; proving the hash relation directly would force a pairing and a hash into a SNARK circuit over BLS12-381, the curve ICP’s vetKD fixes and the one with no cheap 2-chain. We avoid it: instead of proving the relation, we open a random, below-threshold subset of an honest secret sharing, where on the opened shares the hash is simply recomputed in the clear.

It helps to be exact about what $N = \ell$ buys, since it is easy to overstate. It is *not* recoverability — $\omega \bmod \ell$ is a usable Ed25519 scalar under any modulus. It is (i) that $S = s \cdot G$ is an *exact* identity: under $N = q$ it would read $S = s \cdot G - k \cdot (q \bmod \ell) \cdot G$ for an unknown carry $k \in \{0, 1\}$, which breaks the free equation and leaks a biased function of ω across ciphertexts; and (ii) that $c \mapsto \omega$ given s is a bijection, which the share-consistency check below needs in order to pin each share to a single value.

The proof is a publicly verifiable secret sharing of ω : a Feldman⁵ sharing on *both* FCMP++ bases, each share encrypted to the same statement, made honest by a Fiat–Shamir cut-and-choose. Write G and B for the two bases the spend proof checks the offset against⁶, and $Q_i = \omega \cdot G$, $Q_i^B = \omega \cdot B$ for the offset’s two adaptor points.

On the dispute path recovery is cheap and robust. The recipient already holds the $t - 1$ opened shares in the clear — they are part of the proof — so once the arbiter releases σ_m it decrypts unopened ciphertexts, $\omega_k = (c_k - H_F(e(\sigma_m, U_k))) \bmod \ell$, until *one* passes $\omega_k \cdot G = \sum_j k^j F_j$; that is the t -th consistent point. It Lagrange-interpolates $\omega = f(0)$ and confirms

⁵The both-bases sharing and the cut-and-choose follow the timed-release line of VTS (Thyagarajan et al., CCS 2020) and PayMo (assets/2020-1441.pdf).

⁶ B is the re-randomized key-image generator, not a nothing-up-my-sleeve point. Which base is fixed — the key-image generator I itself or a per-broadcast re-randomization — and how the offset composes with the FCMP++ SA+L spend proof is an open design item; the proof binds whichever B is fixed at update time by committing F_j^B against it.

ProveEncryptedOffset. Inputs ω, m, Z, Q_i, Q_i^B ; parameters (n, t) .

- Derive every random choice below from a PRF keyed on (ω, m, Z) , so re-running the prover reproduces the **same** proof byte-for-byte; a second proof over the same ω that opened a *different* subset would leak it.
 - Choose a degree- $(t-1)$ polynomial $f(x) = \sum_{j=0}^{t-1} a_j x^j$ over $\mathbb{Z}_\ell$ with $a_0 = \omega$; the shares are $\omega_k = f(k), k = 1, \dots, n$.
 - Publish Feldman commitments to the coefficients on both bases, $F_j = a_j \cdot G$ and $F_j^B = a_j \cdot B$ for $j = 0, \dots, t-1$. By construction $F_0 = Q_i$ and $F_0^B = Q_i^B$, and each share point $\sum_j k^j F_j$ is then determined.
 - For each coefficient attach a Chaum–Pedersen $\text{DLEQ}_{G,B}(F_j, F_j^B)$, proving F_j and F_j^B carry the same exponent a_j .
 - For each share form $(U_k, c_k) = \text{EncryptToStatement}(\omega_k, m, Z)$, sampling a fresh r_k uniformly in $\mathbb{Z}_q^*$. This is the only pairing work, done in the clear.
 - Compute the cut-and-choose challenge $I = H_{\text{FS}}(\text{dst}, m, Z, B, Q_i, Q_i^B, \{F_j\}, \{F_j^B\}, \{U_k\}, \{c_k\}, n, t)$ as a $(t-1)$ -element subset of $\{1, \dots, n\}$, and open it: reveal (r_k, ω_k) for each $k \in I$.
 - Emit all n share-ciphertexts $\{(U_k, c_k)\}$, the commitments $\{F_j\}, \{F_j^B\}$ with their DLEQs, the subset I , and the openings $\{(r_k, \omega_k)\}_{k \in I}$. The $n - (t-1)$ unopened ciphertexts are the verifiably encrypted offset.
-

Figure 3: Produce a verifiably encrypted offset with its binding proof.

$\omega \cdot G = Q_i$. Because the opened shares and the target Q_i are already verified, a single good unopened ciphertext suffices, and no erasure-coding layer is needed.

Note

VerifyEncryptedOffset

- Target binding.** Check $F_0 = Q_i$ and $F_0^B = Q_i^B$; this ties the sharing to the exact adaptor points **unconditionally**.
 - Both bases.** Check every $\text{DLEQ}_{G,B}(F_j, F_j^B)$.
 - Challenge.** Recompute I from the transcript and confirm it matches the opened set.
 - Opened shares.** For each $k \in I$ recompute $U_k = r_k \cdot G_2$, $y_k = e(H_P(m), Z)^{r_k}$, $s_k = H_F(y_k)$, and verify $c_k = (\omega_k + s_k) \bmod \ell$, $\omega_k \cdot G = \sum_j k^j F_j$, and $\omega_k \cdot B = \sum_j k^j F_j^B$.
 - Point hygiene.** Reject unless every received Ed25519 point $(Q_i, Q_i^B, \text{all } F_j, F_j^B)$ is non-identity and torsion-free ($\ell \cdot P = 0$); recompute B locally rather than trust it on the wire; require canonical scalars (reject any c_k that is not the least residue modulo ℓ); subgroup-check every $U_k \in \mathbb{G}_2$, the unopened ones included; and reject any repeated U_k .
 - Accept iff every check passes.
-

Figure 4: Verify a binding proof.

What the proof rests on. Target binding ($F_0 = Q_i$) is unconditional. Hiding is information-theoretic in the sharing — the $t - 1$ opened shares lie below the reconstruction threshold and are independent of ω , and Feldman reveals only $a_j \cdot G$, already implied by Q_i — and otherwise rests on the discrete-log hardness the adaptor point $Q_i = \omega \cdot G$ assumes anyway. Soundness is *statistical*: a dishonest bundle passes only if the cut-and-choose opens exactly the consistent $(t - 1)$ -subset while every unopened share is inconsistent — probability $1/\binom{n}{t-1}$, or $q_H/\binom{n}{t-1}$ against a prover that grinds the Fiat-Shamir challenge. The payoff of a surviving cheat is a *frozen* update, one whose offset cannot be decrypted at dispute time, never the completion of a *wrong* offset. The assumptions are exactly those the arbiter already carries — BF-IBE / BDH in the random-oracle model — plus Fiat-Shamir: no trusted setup, no SNARK, no cross-group DLEQ, and Figure 1 is reused verbatim.⁷

Three details close the construction.

- **Soundness budget.** Take $(n, t) = (104, 53)$, so the opened subset has $t - 1 = 52$ shares and $\binom{104}{52} \approx 2^{100}$. The margin is deliberately generous: because a surviving cheat only *freezes* one update rather than stealing, that much work to force a single dispute to fail is beyond any rational attacker. A channel wanting smaller messages can lower n ; one wanting more margin can raise it.
- **The artifact is per-update.** The offset and its proof travel on *every* update, not only on the dispute path, so the 23,641 bytes at these parameters (Table 1) are a standing per-update cost, not a rare one. If it ever dominates, the constant-size McFly-style algebraic variant is the documented upgrade, at the price of a re-architected ciphertext and chunked decryption.
- **H2F concretely.** Instantiate `H2F` as a wide reduction — read 64 uniform bytes (SHA-512, or RFC 9380 `expand_message` with $L = 48$) as an integer modulo ℓ — of a Grease-unique domain-separation tag prepended to the canonical fixed-length $\mathbb{G}_T$ serialization of y . The wide reduction holds s within 2^{-128} of uniform on $\mathbb{Z}_\ell$, so $c = \omega + s$ is a sound one-time pad; the unique tag separates this hash from vetKD’s own hashes of pairing outputs; and both parties must reproduce the same $\mathbb{G}_T$ serialization (Section 4.10), since a divergence there stays invisible until decryption fails on the dispute path.

Because the cost is paid on every update, the size is worth stating exactly rather than approximately. Write π for the encoded proof. Its canonical encoding has no variable-length field: a nine-byte header (a version byte, then n and t as 32-bit little-endian integers), the two commitment vectors, one DLEQ per coefficient, all n share ciphertexts in index order, and the openings in ascending index order. The subset I is not sent separately — each opening carries its own index — and B is not sent at all, since the verifier recomputes it. Every element is fixed-width, so the length is exactly⁸

⁷A one-page game-based reduction, and the party-side vetKD reproduction vectors of Section 4.10, are tracked as follow-ups before deployment.

⁸The implementation pins this figure: `BindingProofParams::encoded_len` in `libgrease/src/cryptography/binding_proof.rs`, with a regression test that recomputes the layout independently and asserts 23,641 bytes at the production profile.

$$|\pi| = \underbrace{9}_{\text{header}} + \underbrace{128n}_{\text{ciphertexts}} + \underbrace{128t}_{\text{commitments + DLEQs}} + \underbrace{68(t-1)}_{\text{openings}} \text{ bytes}, \quad (6)$$

which at $(n, t) = (104, 53)$ is **23,641 bytes**, or 23.1 KiB; Table 1 breaks it down. The share ciphertexts are 56% of that, and inside each of them it is the 96-byte compressed $\mathbb{G}_2$ point, not the 32-byte Ed25519 scalar, that sets the scale. A rough estimate that prices the $\mathbb{G}_2$ encodings at Ed25519 width lands near 20 KB and under-budgets the update path by about a fifth.

COMPONENT	ELEMENT	COUNT $\times$ WIDTH	BYTES
Header	version $\parallel n \parallel t$	1×9	9
Commitments	F_j, F_j^B	$2t \times 32$	3,392
Coefficient DLEQs	challenge $\parallel$ response	$t \times 64$	3,392
Share ciphertexts	$U_k \parallel c_k$	$n \times (96 + 32)$	13,312
Openings	$k \parallel r_k \parallel \omega_k$	$(t - 1) \times (4 + 32 + 32)$	3,536
Total			23,641

Table 1: The canonical binding-proof encoding at $(n, t) = (104, 53)$.

4.6. The dispute state machine

The arbiter keeps a small amount of **non-secret** state per channel: a monotonically increasing **high-water mark**, an **adjudication window** anchored to a consensus clock, a **resolved tombstone**, and a public append-only **action log**. When a party presents a cross-signed record, the arbiter runs the lifecycle shown in Figure 5.

The record a party presents is the same object the parties sign on *every* update:

```
pub struct UpdateRecord {
    /// The globally unique channel id (commits to the funding output)
    channel_id: ChannelId,
    /// The monotonic update count for this state
    update_count: u64,
    /// The canonical signable hash of state i's closing transaction
    close_hash: Hash,
    /// This record signed by the customer, P_A
    signature_a: SchnorrSignature,
    /// This record signed by the merchant, P_B
    signature_b: SchnorrSignature,
}
```

Verifying an `UpdateRecord` is two Schnorr checks and a well-formedness check on the record itself — a valid `channel_id`, an update count, and a present `close_hash`. That both parties signed the record is what vouches that `close_hash` is the legitimate closing transaction for this state; the arbiter does not open the id or reconstruct the transaction, so it never learns the funding output, the linking tag, or the balances. It sees only an opaque id and an update count.

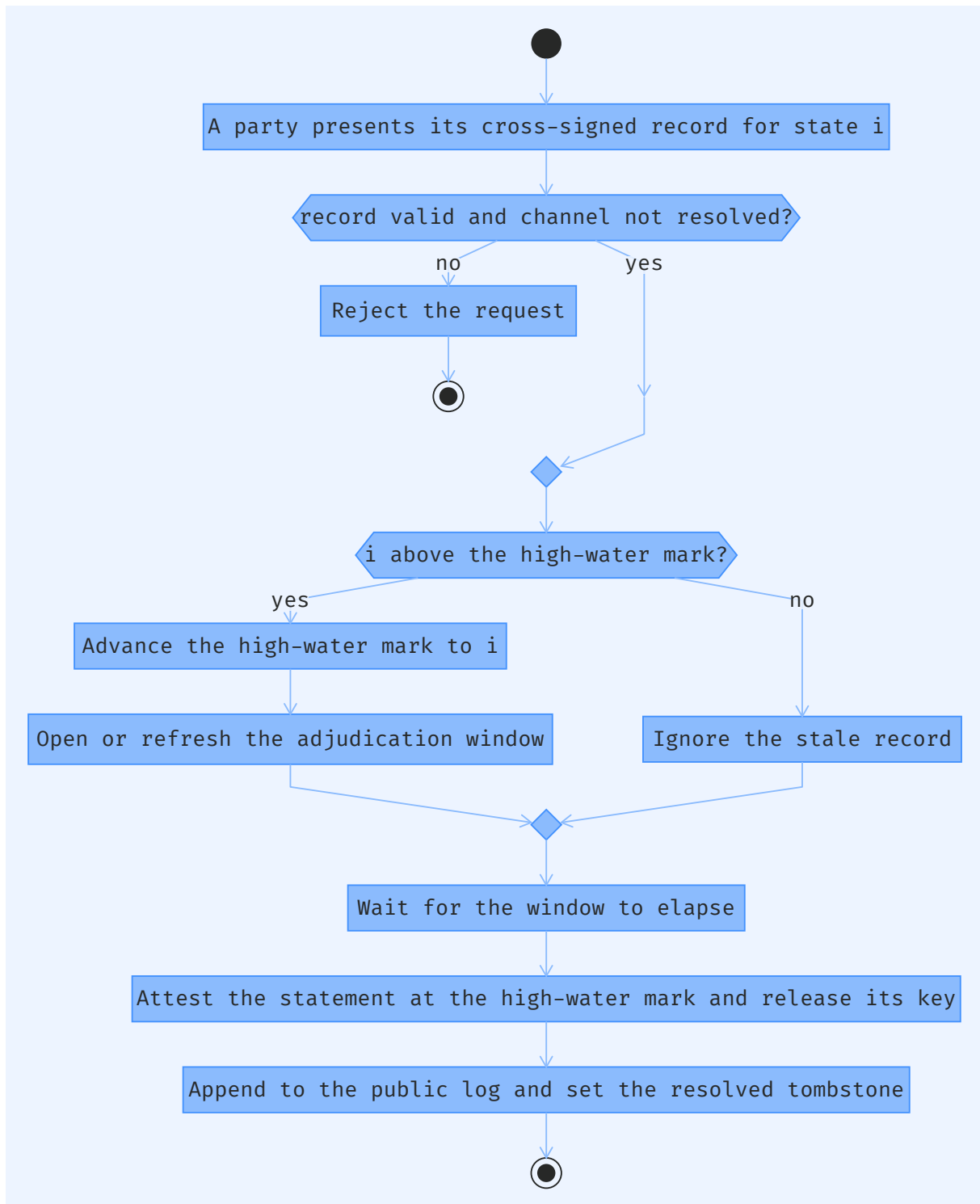


Figure 5: The arbiter’s per-channel lifecycle. The high-water mark only ever rises, so the statement finally attested is the maximum record anyone presented.

When the window elapses with no higher record, the arbiter attests the high-water statement and logs it:

Two rules keep the attestation honest and useful. First, **the window anchors to the platform’s consensus clock or a beacon round, never to a host’s wall-clock** — a single operator that controlled a clock could otherwise freeze the window or fast-forward through it, whereas consensus time cannot be moved by a minority below the fault threshold. Second, **the attestation binds the specific close it authorizes**, and it does so without the arbiter

```
onPresentRecord(rec: UpdateRecord)
```

```
1 Validate signature_a and signature_b over the record, and that rec is well-formed (a valid
   channel_id, an update count, and a present close_hash). If not, return Fail(Invalid).
2 Fetch the DisputeState for rec.channel_id; create one if absent.
3 if state.resolved: return Fail(AlreadyResolved).
4 if rec.update_count > state.high_water:
5     state.high_water = rec.update_count
6     state.window_expiry = now() + dw (anchored to the platform's consensus time or a beacon
   round)
7     append (RecordPresented, rec.update_count) to state.log
8 else: append (StaleRecordIgnored, rec.update_count); take no further action.
9 Return Ok.
```

Figure 6: The arbiter's response to a presented record.

```
onWindowElapsed(id: ChannelId) — self-triggered by an on-chain timer
```

```
1 Fetch the DisputeState for id; if state.resolved, return.
2 if now() < state.window_expiry, return (not yet).
3 Let  $\bar{i} = \text{state.high\_water}$  and  $m = (\text{id}, \bar{i})$ .
4 Compute the attestation  $\sigma_m = z \cdot H_P(m)$  under the threshold key and publish it.
5 Append (Attested, high_water) to state.log; set state.resolved = true.
```

Figure 7: Resolving a dispute at the window's close.

inspecting the transaction: m_i names `id`, which commits to the funding output (Section 3.3.1), and the offset it unseals is bound at update time to the adaptor point of *this* state's closing signature (its binding proof). So even though the released σ_{m_i} is public, it completes nothing but the one close it names — a stale or foreign transaction has no matching offset to complete.

4.7. The dispute in practice

Honest unilateral close. The counterparty has gone silent. The honest party presents the record for the latest state n ; the arbiter sets the high-water mark to n and opens the window; no higher record arrives; at the window's close the arbiter attests m_n . The honest party decrypts the counterparty's offset ω_n (Figure 2), completes the counterparty's half of the closing signature, and broadcasts the state- n close. The window is the dispute delay — measured in hours, configurable per channel — and the counterparty's balance is paid to its own address regardless.

Cheating is frozen. Suppose a party instead presents a stale record for state i , hoping to close at a more favorable old balance. The arbiter opens the window at high-water i . The honest party — watching the arbiter's log, or delegating that watch to a third party — answers with the record for the true latest state $n > i$. The high-water mark advances to n and the window refreshes; at its close the arbiter attests m_n , **not** m_i . The honest party completes the state-

n close; the cheat’s ciphertext for m_i is never unsealed and its stale close stays frozen. The cheater gains nothing and the honest party loses nothing.

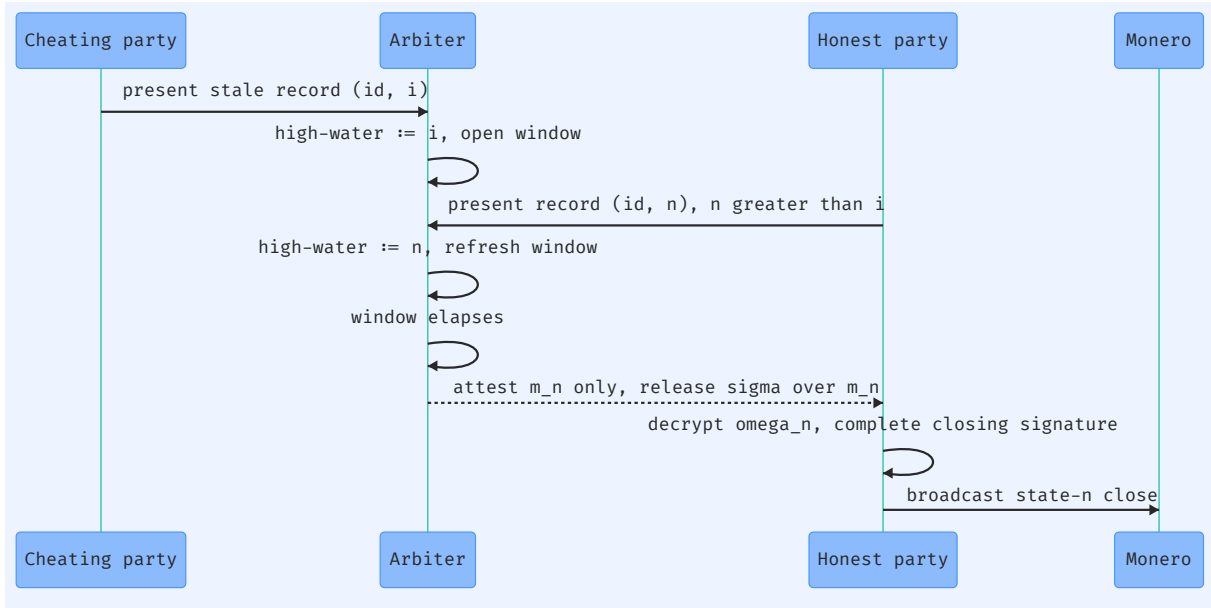


Figure 8: A stale close is frozen, not punished: the arbiter attests only the maximal presented record.

Watching duty. Because a dispute is resolved off-chain, a party must watch the arbiter’s log for disputes on its channels, just as it watches the Monero chain for a close. Both duties are delegable to a watchtower, which needs only the channel’s funding-output linking tag and the presentable records; it can never redirect funds, since a close pays the parties’ own addresses. Handing the linking tag to a watchtower grants it exactly one capability — to *recognize* this channel’s close when it lands, and nothing else (Section 4.8).

4.8. What is public, and to whom

The design leaks strikingly little, but “little” is not “nothing”, and it is worth being exact about what the general public, a watchtower, and the arbiter chain can each learn.

A completed close on Monero an ordinary transaction. To anyone who does not already hold the funding output’s linking tag L_F , it is indistinguishable from any other spend — one-time stealth addresses, amounts hidden by confidential transactions, and under FCMP++ the spend proven against the whole membership set, publishing only L_F as the double-spend tag and never revealing *which* output was consumed. Monero’s base-layer privacy is entirely undisturbed by the arbiter.

Holding L_F a *watch capability* and nothing more. The linking tag is deterministic in the funding output’s key and — this being the whole point of a Monero key image — cannot be linked back to that output by anyone who does not already hold it. So whoever has L_F can recognize the one transaction that closes this channel, and learn nothing else: not the balances, not the addresses, not the identities. The parties compute L_F jointly and may delegate it to a watchtower for exactly this recognition duty.

The arbiter chain only `id` (which *commits* to L_F without revealing it), an update count, and a close-transaction hash — never L_F , the balances, or the addresses. A bystander

therefore cannot find the Monero close from the arbiter chain: `id` is opaque, and without L_F one cannot scan for it. The one residual leak is dispute-path metadata — that some opaque channel was adjudicated, and the update count it reached — carrying no identities and no link to Monero, and never arising on the cooperative path.

Note

If even that metadata is too much, two independent hardenings are available, neither on by default. The update count need only be *monotonically increasing*, not a strict increment, so states can be numbered sparsely ($i \in \{0, 1, 135, 34873, \dots\}$) and the high-water mark then reveals only the last label reached, not how many updates occurred. And where a deployment needs an external verifier (a watchtower or auditor) to confirm that a given close matches a given record *without being handed the opening*, the id's commitment to L_F (Section 3.3.1) can use a proof-friendly scheme such as a Pedersen commitment, so that binding is provable in zero knowledge; the id is already blinded by its per-channel nonces, so this buys provability, not hiding.

4.9. Trust and collusion

Everything the arbiter guarantees rests on **two platform boundaries**. The first is the validator set staying below its Byzantine-fault-tolerance threshold: below it, the published state machine (and only it) executes, and the master key z stays confidential. The second — quieter, and often *lower* — is governance over the subnet that actually holds z : whoever can reshare that key onto a new node set, or replace the code that runs beside it, inherits the arbiter's authority without crossing the first threshold (the upgrade path below). What there is *not* is a second, channel-specific secret, because the design has deliberately left none.

Consider the worst case: enough of the committee is corrupt to attest an arbitrary statement, or the master key z leaks outright. What can an attacker do? Attest some m_i for a stale state and hand the released key to a colluding party, who completes that state's close. That is the whole attack surface. It cannot fabricate a state (there is no verifiably encrypted offset and no cross-signed record for a state the parties never built), cannot alter balances (the close template is fixed at update time), and cannot reach any channel where it lacks a colluding party (completing a close still needs that party's own funding signature). The reachable damage is a rollback to **some past state both parties actually signed** — never an arbitrary theft. On the honest execution path it is also **public and attributable**: every honest attestation is logged before it is usable, and every completed close appears on-chain against L_F — in view of whoever holds it (the parties and any watchtower they appointed; Section 4.8), against which a release that does not match the deterministic high-water logic stands out as a deviation. This loudness is a property of honest execution rather than an unconditional one: because the platform delivers the attestation encrypted to the requester rather than broadcasting it, a committee already corrupt enough to sign — or a captured governance layer — can derive a key off-chain and bypass the log. The guarantee thus deters an honest-but-curious operator and records the honest path faithfully; it does not by itself bind an attacker at or above the threshold. Even so it improves on a custody-based escrow, whose bad release is an *off-protocol*

handover that leaves no trace on *any* path, detectable only by the fraud-proof-and-bond machinery such a design must add for the purpose.

⚠ Caution

The upgrade path is the sharp edge. The arbiter’s honesty is the honesty of its code, so whoever can *change the code* inherits its authority. If the deployed program can be replaced while keeping the same on-chain identity — and therefore the same derived attestation key — an operator could swap in logic that attests any statement it likes. Removing that path — making the program immutable once fixed, or placing control under a decentralized governance body rather than a single operator — is necessary but not sufficient on its own. Because the threshold key is held by the platform’s validator subnet, not by the program, freezing the program does not remove the platform’s own power to reshare that key onto a new node set or change the code running beside it. Governance over the key-holding subnet must therefore be decentralized and audited too; otherwise a deployment has not removed the trusted secret, only moved it up a layer.

FAILURE	CONSEQUENCE
Arbiter honest, one party cheats	Stale close frozen; honest close proceeds. No loss.
Arbiter halts (liveness only)	Honest close waits for cooperation; funds are never at risk. A time-beacon extension could restore liveness at a penalty-channel cost (Section 4.12).
Committee compromised and a channel party colludes	Rollback to a <i>past party-signed state</i> on that channel; on-chain, and logged on the honest path, but a threshold-compromised committee can derive the key off-chain.
Committee compromised, no colluding party	Nothing on that channel — the funding signature is still missing.
Deployment or platform governance captured	Full arbiter authority — the reason to decentralize governance over the key-holding subnet, not only the program.

Table 2: What each failure mode costs.

4.10. Instantiation on the Internet Computer

The Internet Computer’s *vetKeys* feature (exposed as *vetKD*, verifiably encrypted threshold key derivation) is a production realization of exactly the primitive Section 4.3 needs: a validator set that holds a stable threshold-BLS master key on curve `bls12_381_g2` and releases a signature on a caller-supplied, application-chosen byte string, gated by canister logic, and delivered blinded so that only the requester learns it.

- The management-canister method `vetkd_public_key` returns the stable master (or context-derived) public key Z for a triple `(canister, context, key_id)`. The arbiter fixes a `context` and publishes the resulting Z ; offsets are encrypted against it.
- `vetkd_derive_key` releases the key for a caller-supplied `input` — here `input = m_i` — encrypted under a caller-supplied transport key. The canister calls it only when its own state machine has reached “attest the high-water statement” (Figure 7), so the on-chain logic is the gate.
- Canister *timers* let the state machine self-trigger the window’s expiry, so Figure 7 runs autonomously without an external poker.
- The `ic-vetkeys` libraries provide the threshold-signing helpers, and `vetkd_derive_key` yields a genuine, pairable BLS signature σ_m . Its shipped identity-based-encryption path, however, is a byte-oriented hybrid scheme, so the verifiably encrypted offset is not that object: it is a purpose-built encryption over the released σ_m that matches Figure 1–Figure 2 and admits the binding proof.
- The optional fee-and-deposit bond (Section 4.11) falls out for free: a canister holds and moves value natively (its own cycles or an ICRC-1 token), so registration can gate on an ICRC-1 transfer of $d_k + f_k$ from B , the fee is credited to the committee’s treasury, and the same deterministic state machine that attests the high-water statement also pays the deposit to the winner (or refunds it to B on a cooperative-close cleanup call). No new trust is introduced — only the vetKD attestation is security-critical, and it is untouched.

Note

Two integration details carry real weight. The offset is encrypted on the Monero/Ed25519 side and decrypted with a BLS12-381 key, so the group placement, point serialization, and especially the hash-to-curve derivation — which augments the hashed statement with the derived public key — must be reproduced exactly by the party-side encryptor. And the deployment must fix its upgrade path, and decentralize governance over the key-holding subnet, before it holds real value.

The two boundaries of the previous section fall, for ICP, like this: the key is held by the *subnet* the canister runs on — on the order of 34 nodes, with forging a signature requiring a two-thirds quorum — and the *Network Nervous System* governs that subnet’s membership, its replica code, and the re-sharing that keeps the key alive, at a threshold that can sit below the subnet’s own. Closing that second boundary — decentralizing the governance, or accepting an immutable canister and its loss of patchability — is the deployment’s real work. ICP is not the only platform that could host the arbiter; Fairblock, dcipher and Sui’s Seal expose comparable threshold identity-based encryption. It is the turnkey option: it custodies value and self-triggers natively, and it is live on mainnet with a completed external audit.

4.11. Optional: fees, deposits, and cleanup

This mechanism is *optional*: a deployment that wants the arbiter to hold no value at all omits it. Where it is used, it lets the arbiter charge for its service and gives the parties a bond that makes the cooperative path the cheapest one.

At channel setup, a *deposit* $d_k = d_a + d_b$ and a *fee* f_k are lodged with the arbiter contract in the arbiter chain's own currency. On the common paths only the merchant B touches that currency: B fronts the whole $d_k + f_k$, and the customer A reimburses her share $d_a \leq d_k$ across the Monero channel as an ordinary in-channel payment. So each party bears its own portion of the bond while A need not hold the arbiter chain's token to open a channel. The arbiter keeps f_k as revenue and holds d_k until the channel resolves, releasing it as follows:

Cooperative close (the common path) B presents the cooperatively-signed final state — which A signed, and which already accounts for the d_a paid through the channel — and the arbiter refunds d_k to B and tombstones the channel. Reclaiming the bond is exactly what incentivizes B to return and clear per-channel state rather than leave stale disputes cluttering the arbiter chain.

Adjudicated close the arbiter pays d_k to the *winner* — the party that presented the maximal record — at the window's close. Because the winner is always the honest closer, whoever forced the dispute (by cheating or by going silent) forfeits their share to the party that behaved. This is a mild, bounded nudge toward cooperation, not a capacity-sized penalty.

Note

Taking part in a dispute — presenting a higher record, or claiming the deposit as winner — does require A to talk to the arbiter chain and receive a payout on it. The dispute path is already interactive and rare, so an A that anticipates needing it keeps a minimal arbiter-chain presence or delegates both the watch and the claim to a watchtower, which can act for her without ever redirecting channel funds.

The bond is *value on the arbiter chain, not a channel secret*. A compromised arbiter can at worst misdirect a bond — an amount bounded by d_k , chosen small, and visibly on the arbiter chain — never the Monero funds, which still need a party's own funding signature and the true offset. The custody-free property concerns channel secrets, of which the arbiter still holds none; a small, bounded, on-chain bond is a categorically smaller exposure.

4.12. Possible extensions

The core design is deliberately minimal, and two directions extend it — each buying something real at a cost worth stating before it is adopted.

A time-beacon liveness backstop An arbiter halt costs liveness (Table 2): an honest close waits until the counterparty cooperates. To survive a halt *without* the counterparty, each verifiably encrypted offset can be composed so that it opens under *either* condition — the arbiter attests m_i , **or** a public randomness beacon reaches a pre-agreed round. The price, though, is the whole price of a penalty channel. A time branch cannot know which state is latest, so it opens *every* state's offset, stale ones included — and the offset is the same object whether its state turns out latest or stale, fixed when the ciphertext is sealed. A patient cheater need only wait for an old state's beacon round to pass and then race an honest close onto the chain; making that race safe pulls in the full standalone regime — collateral roughly equal to the channel capacity, a bounded channel lifetime, and a regating schedule so the latest state matures first. Beacon openings are also silent, leaving

no entry in the arbiter’s log. It is a heavyweight option, suited only to channels that must survive an arbiter halt and can afford capacity-sized collateral; the default deployment omits it.

Proof-friendly commitments and sparse numbering The privacy hardenings of Section 4.8 are extensions in the same sense — numbering states sparsely hides the update count, and a proof-friendly commitment of `id` to L_F lets an external verifier check a close against a record in zero knowledge. Neither is on by default, and neither changes the core safety argument.

4.13. Summary

The arbiter is a deterministic dispute state machine that holds no share of the funding key and no per-channel secret. It keeps a high-water mark, an adjudication window, a tombstone, and a public log, and its only power is to attest one statement — the true latest state — which unseals exactly the offset that closes at it. A stale close is frozen rather than punished, so cheating cannot pay. Trust reduces to two platform boundaries — the validator set’s fault threshold and governance over the key-holding subnet; the worst residual failure is a bounded rollback to a past state both parties signed, public and attributable on the honest execution path, and only with a colluding channel party. It leaks strikingly little — the arbiter never sees balances, addresses, or the funding output’s linking tag in the clear, and a bystander cannot link its opaque records to a Monero close. An optional fee-and-deposit bond funds the service and rewards cooperation without ever holding a channel secret, and a time-beacon extension (Section 4.12) can trade capacity-sized collateral for survival of an arbiter halt. Instantiated on a consensus-held threshold-BLS key, with the deployment’s upgrade path closed, the arbiter gives Grease fair dispute resolution without ever asking anyone to hold a secret whose leak would matter.

5. Future extensions and known limitations

Grease assumes that Monero will never implement state management features such as time-locks and scripting capabilities — the very features that would let a channel enforce its own dispute resolution on the base layer. Lacking them, Grease delegates that role to an external arbiter, and its principal limitations follow from that separation.

5.1. Bilateral channels only

Grease is a two-party channel and does not attempt multi-hop routing — an explicit anti-principle of the design. Taking the Lightning Network as a case study, we hold that the bulk of a payment channel’s utility is captured by bilateral channels at a small fraction of the complexity. Networks built by composing Grease channels are possible, but are not a goal of this work.

5.2. Trust in the arbiter

Grease strips custody from its dispute resolver but not authority: the arbiter’s guarantees rest on the honesty of its consensus platform. The worst-case failure is a bounded rollback to a past state *both parties actually signed*, and only when the committee is compromised **and** a channel party colludes; the funding key is never at risk and balances can never be fabricated. The full trust analysis is set out in Section 4.

5.3. Liveness depends on cooperation or an extension

If the arbiter halts, an honest party can still close, but only with its counterparty’s cooperation; funds are never at risk, but a unilateral close must wait. A channel that must survive an arbiter halt *without* cooperation can adopt the time-beacon backstop of Section 4.12, at the price of capacity-sized collateral and a bounded lifetime — which is why it is off by default.

5.4. Watching duty

Because a dispute is resolved off-chain, each party must watch the arbiter’s log for disputes on its channels, just as it watches the Monero chain for a close. Both duties are delegable to a watchtower — with the associated privacy risks — which can recognize and answer a dispute but can never redirect funds, since a close always pays the parties’ own addresses (Section 4.8).

Table of Algorithms

6. Nomenclature

6.1. Symbols

SYMBOL	DESCRIPTION
G	Generator point for the Ed25519 curve
N	The prime order of the Ed25519 group
ω_i	The secret adapter-signature offset for channel state i
Q_i	The public adapter point corresponding to ω_i on Ed25519, $Q_i = \omega_i \cdot G$
m_i	The dispute statement for state i (“on channel id, state i is the latest”)
Z	The arbiter committee’s stable threshold-BLS master public key, $Z = z \cdot G_2$
σ_m	The arbiter’s attestation of a statement m : a threshold-BLS signature that acts as the decryption key for m

6.2. Subscripts

SUBSCRIPT	REFERENT
Ed	Curve Ed25519
M	The peer playing the role of merchant
C	The peer playing the role of customer
Initiator	The peer playing the role of initiator
Responder	The peer playing the role of responder

References

1. What is Monero?. <https://www.getmonero.org/get-started/what-is-monero/>[◦]
2. Chainalysis. All About Monero. <https://www.chainalysis.com/blog/all-about-monero/>[◦]
3. Alexander Culafi. Monero and the complicated world of privacy coins. January 24, 2022. <https://www.techtarget.com/searchsecurity/news/252512394/Monero-and-the-complicated-world-of-privacy-coins>[◦]
4. Zhimei Sui, Joseph K. Liu, Jiangshan Yu, Man Ho Au, Jia Liu. *Auxchannel: Enabling Efficient Bi-Directional Channel for Scriptless Blockchains.*; 2022. <https://eprint.iacr.org/2022/117.pdf>[◦]
5. Sui Z, Liu JK, Yu J, Qin X. *Monet: A Fast Payment Channel Network for Scriptless Cryptocurrency Monero.*; 2022. <https://eprint.iacr.org/2022/744.pdf>[◦]
6. CJ, Sean Coughlin. Grease: A minimal Monero Payment Channel implementation for Monero. <https://cfp.twed.org/mk5/talk/QYDGPM/>[◦]
7. jeffro256. Personal communication: Signature changes post-FCMP. Published online June 22, 2025.

Index of Figures

Figure 1	Encrypt an offset to a statement.	23
Figure 2	Decrypt an offset once the statement has been attested.	23
Figure 3	Produce a verifiably encrypted offset with its binding proof.	25
Figure 4	Verify a binding proof.	25
Figure 5	The arbiter’s per-channel lifecycle. The high-water mark only ever rises, so the statement finally attested is the maximum record anyone presented.	29
Figure 6	The arbiter’s response to a presented record.	29
Figure 7	Resolving a dispute at the window’s close.	29
Figure 8	A stale close is frozen, not punished: the arbiter attests only the maximal presented record.	31

Index of Tables

Table 1	The canonical binding-proof encoding at $(n, t) = (104, 53)$	28
Table 2	What each failure mode costs.	33

Index of Listings

Listing 1	The New Channel proposal sequence	11
Listing 2	Establishing a new Channel	13
Listing 3	Establishing a new Channel, continued	14
Listing 4	Creating a new multisig wallet	16
Listing 5	Updating a Channel	18
Listing 6	The Channel Close sequence	20